



SQL for Data Science: Quick Revision Notes

These notes are a structured and translated compilation of the SQL course from Coursera. They are formatted for quick revision and easy reference.

1. SQL Basics

SQL (Structured Query Language)

- **Full form:** Structured Query Language
- **Use:** A language used to query or retrieve data from relational databases.

What is Data?

- A collection of facts in the form of words, numbers, or pictures.
- One of the most critical assets for any business.

What is a Database?

- A program or repository used to store data.
- Provides the functionality to add, modify, and query data.

Relational Database

- A database where data is organized in a tabular form (columns and rows, similar to a spreadsheet).
- Columns contain the properties of an item (e.g., `LastName`, `FirstName`, `City`).
- Relationships can be established between different tables.

DBMS & RDBMS

- **DBMS (Database Management System):** A set of software tools used to manage the data within a database.
- **RDBMS (Relational Database Management System):** A set of software tools specifically for relational databases (acts as the backbone for applications in banking, health, transportation, etc.).

- **Examples:** MySQL, Oracle Database, DB2 Warehouse, DB2 on Cloud.

5 Basic SQL Commands

1. **Create:** To create a table.
 2. **Insert:** To populate/fill data into a table.
 3. **Select:** To view/retrieve data.
 4. **Update:** To modify existing data.
 5. **Delete:** To remove data.
-

2. SELECT Statement & Data Retrieval

SELECT Statement

- A **DML (Data Manipulation Language)** statement used to read or retrieve data.
- It is also referred to as a query, and the output generated from its execution is called a **result set** or **result table**.

Basic Syntax

- **All columns:** *(Often read as "select star from table name")*

```
SELECT * FROM table_name;
```

```
* **Specific columns:** *(Note: Columns are displayed in the exact order  
they are written in the query).*
```

```
```sql
```

```
SELECT column1, column2 FROM table_name;
```

### WHERE Clause & Predicate

- The **WHERE** clause is used to filter and restrict the result set.
- **Predicate:** A condition that evaluates to true, false, or unknown.

```
SELECT book_id, title FROM book WHERE book_id = 'B1';
```

## Comparison Operators (Used within the WHERE clause)

1. Equal to ( = )
  2. Greater than ( > )
  3. Less than ( < )
  4. Greater than or equal to ( >= )
  5. Less than or equal to ( <= )
  6. Not equal to ( <> or != )
- 

## 3. SQL Useful Expressions (COUNT, DISTINCT, LIMIT)

### COUNT Function

- **Purpose:** Returns the total number of rows (count) that match the query criteria.
- **Syntax & Examples:**
- Count all rows:

```
SELECT COUNT(*) FROM tablename;
```

- Count with a condition:

```
SELECT COUNT(COUNTRY) FROM MEDALS WHERE COUNTRY = 'CANADA';
```

### DISTINCT Keyword

- **Purpose:** Removes duplicate values from the result set, displaying only unique values.
- **Syntax & Examples:**
- Retrieve unique values:

```
SELECT DISTINCT columnname FROM tablename;
```

- List of unique countries that won gold medals:

```
SELECT DISTINCT COUNTRY FROM MEDALS WHERE MEDALTYPE = 'GOLD';
```

## LIMIT Clause

- **Purpose:** Restricts the number (quantity) of rows retrieved from the database.
- **Syntax & Examples:**
- View the first 10 rows:

```
SELECT * FROM tablename LIMIT 10;
```

- Apply limit on filtered data:

```
SELECT * FROM MEDALS WHERE YEAR = 2018 LIMIT 5;
```

---

## 4. INSERT Statement

- **Purpose:** Used to add (populate) new rows of data into a table.
- **Type:** A DML (Data Manipulation Language) statement.

### Basic Syntax

```
INSERT INTO table_name (column1, column2, column3) VALUES (value1, value2, value3);
```

### Key Rules

1. **Column & Value Match:** The total count of values in the `VALUES` clause must be exactly equal to the number of columns listed.
2. **Order Matters:** The values provided must strictly follow the same sequence as the specified columns.

## Row Insertion Methods (2 Methods)

1. **One row at a time:** Inserting a single record.
2. **Multiple rows at a time:** Inserting multiple records in a single `INSERT` statement by separating value sets with a comma ( , ).

```
INSERT INTO author (author_id, last_name, first_name)
VALUES ('A1', 'Chong', 'Raul'),
 ('A2', 'Ahuja', 'Rav');
```

---

## 5. UPDATE & DELETE Statements

**Type:** Both are DML (Data Manipulation Language) statements used to modify or remove data.

### 1. UPDATE Statement

- **Purpose:** To alter or modify existing data within a table.
- **Basic Syntax:**

```
UPDATE table_name SET column1 = value1, column2 = value2 WHERE
condition;
```

- **Example:**

```
UPDATE AUTHOR SET LAST_NAME = 'KATTA', FIRST_NAME = 'LAKSHMI' WHERE
AUTHOR_ID = 'A2';
```

⚠ **Critical Warning:** If you forget to include the `WHERE` clause, all rows in the table will be updated!

## 2. DELETE Statement

- **Purpose:** To permanently remove one or more rows from a table.
- **Basic Syntax:**

```
DELETE FROM table_name WHERE condition;
```

- **Example:**

```
DELETE FROM AUTHOR WHERE AUTHOR_ID IN ('A2', 'A3');
```

⚠ **Critical Warning:** If you execute `DELETE FROM table_name;` without a `WHERE` clause, the entire data of the table will be deleted (the table will be emptied)!

---

## 6. Database Concepts

### 1. Relational Model

- The most widely used data model.
- Stores data in a simple structure: Tables (Rows and Columns).
- **Main Advantage:** Provides Data Independence:
  - Logical data independence
  - Physical data independence
  - Physical storage independence

### 2. ER Model (Entity-Relationship Model) & ERD

- Used as a conceptual tool to design relational databases.

- **ERD (Entity-Relationship Diagram):** A visual representation of entities and their relationships.
- **Building Blocks:**
- **Entity:** Any real-world object (Noun: Person, Place, Thing) that exists independently.
- ERD Symbol: Rectangle
- Examples: Book, Author, Borrower
- **Attribute:** The characteristics or properties that provide details about the entity.
- ERD Symbol: Oval
- Examples: Title, Edition, and Year for a Book.

### 3. Mapping (ER Model to Relational Table)

ER Model Component	Relational Database Equivalent
Entity	Table
Attribute	Column
Data Values	Row / Tuple

### 4. Common Data Types

- **CHAR (Character):** For fixed-length text (e.g., ISBN, State codes).
- **VARCHAR (Variable Character):** For variable-length text (e.g., Title, Name, Email).
- **Numeric (INTEGER / DECIMAL):** For numbers (e.g., Edition, Year, Price).
- **Date / Time:** For storing dates and timestamps.

### 5. Database Keys

- **Primary Key (PK):**
- Uniquely identifies every single row (tuple) in a table.
- Prevents data duplication (no duplicates allowed, cannot be null).
- **Foreign Key (FK):**
- A Primary Key from another table that is added to the current table.

- Used to establish a relationship or link between tables.
- 

## 7. Types of SQL Statements

SQL statements are used to interact with Entities (tables), Attributes (columns), and Tuples (rows/data values) in relational databases. They are primarily divided into two categories:

### 1. DDL (Data Definition Language)

- **Purpose:** To define, modify, or delete the structure of database objects (such as tables).
- **Common Statements:**
  - **CREATE** : To create a new table and define its columns/data types.
  - **ALTER** : To modify an existing table's structure (add columns, drop columns, or change data types).
  - **TRUNCATE** : To delete all data inside a table while keeping the table structure intact.
  - **DROP** : To permanently delete the entire table along with its structure.

### 2. DML (Data Manipulation Language)

- **Purpose:** To read and modify the data present inside tables.
- **CRUD operations mapping:**
  - Create → **INSERT**
  - Read → **SELECT**
  - Update → **UPDATE**
  - Delete → **DELETE**
- **Common Statements:**
  - **INSERT** : Adds one or multiple rows to a table.
  - **SELECT** : Reads or fetches rows/data from a table.
  - **UPDATE** : Edits or modifies data in existing rows.
  - **DELETE** : Removes specific row(s) from a table.

## SQL Statement Categories Comparison

Category	Main Focus	Target	Common Commands
DDL	Defining the Structure / Schema	Database Objects (Tables)	CREATE, ALTER, TRUNCATE, DROP
DML	Manipulating the Data	Rows / Records	INSERT, SELECT, UPDATE, DELETE

## 8. CREATE TABLE Statement

- **Type:** The most common DDL (Data Definition Language) statement. Used to create new tables (Entities) and their columns (Attributes) in the database.

### 1. Basic Syntax Structure

```
CREATE TABLE table_name (
 column1_name data_type [constraints],
 column2_name data_type [constraints],
 column3_name data_type [constraints]
);
```

- Columns are defined inside parentheses ( ) following the table name.
- Each column definition is separated by a comma , .

### 2. Column Components

1. **Column Name:** The name of the attribute (e.g., `author_id`, `firstname`).
2. **Data Type:** The format of the stored data:
  - `CHAR(n)` : Fixed-length string (e.g., `CHAR(2)` for country codes like 'AB', 'CA').
  - `VARCHAR(n)` : Variable-length string (up to a maximum of `n` characters, e.g., `VARCHAR(40)` for an email).
3. **Optional Constraints:** Rules applied for data validation.

### 3. Key Constraints Mentioned

- **PRIMARY KEY:**

- Uniquely identifies each row (tuple) in the table.
- Does not allow duplicate records.
- **NOT NULL:**
- Ensures that a blank or NULL value cannot be inserted into the column (e.g., an author's name or ID cannot be left empty).

## 4. Practical Example (Library Database)

```
CREATE TABLE author (
 author_id CHAR(2) PRIMARY KEY NOT NULL,
 lastname VARCHAR(15) NOT NULL,
 firstname VARCHAR(15) NOT NULL,
 email VARCHAR(40),
 city VARCHAR(15),
 country CHAR(2)
);
```

- `author_id`: The primary identifier, hence marked as `PRIMARY KEY NOT NULL`.
- `lastname` and `firstname`: Names are mandatory, hence the `NOT NULL` constraint is applied.
- `email`, `city`, `country`: These are optional fields (without `NOT NULL`), meaning they can be left blank.

---

## 9. ALTER, DROP & TRUNCATE Statements

These are all DDL (Data Definition Language) statements utilized to modify a table's structure or delete tables/data entirely.

### 1. ALTER TABLE Statement

- **Purpose:** To modify the structure of an existing table (e.g., adding/dropping columns, changing data types, or adding/removing keys and constraints).
- **Syntax Note:** Unlike `CREATE TABLE`, it does not use parentheses `( )` to enclose parameters.

- **Operations & Syntax:**
- **Add Column:** Appends a new column to the table.

```
ALTER TABLE author ADD COLUMN telephone_number BIGINT;
```

*(Note: BIGINT can store numbers up to 19 digits long).*

- **Modify Data Type:** Changes the data type of an existing column (e.g., changing from numeric to CHAR to allow + or dashes in a phone number).

```
ALTER TABLE author MODIFY telephone_number CHAR(20);
```

 **Warning (Data Compatibility):** If the column already contains incompatible data (e.g., changing a text column to numeric when it contains non-numeric characters), the database will throw an error and the query will fail to execute.

- **Drop Column:** Removes a column from the table.

```
ALTER TABLE author DROP COLUMN telephone_number;
```

## 2. DROP TABLE Statement

- **Purpose:** To permanently delete a table from the database.
- **Effect:** Deletes both the table structure and all its data simultaneously.
- **Syntax Example:**

```
DROP TABLE author;
```

## 3. TRUNCATE TABLE Statement

- **Purpose:** To delete all rows (data) at once, while keeping the table structure intact.

- **Advantage:** It is much faster and more efficient than using a `DELETE` statement without a `WHERE` clause.
- **Syntax Example:**

```
TRUNCATE TABLE author IMMEDIATE;
```

*(Note: The IMMEDIATE keyword specifies that the statement processes instantly and cannot be undone/rolled back).*

## Quick Comparison: DROP vs TRUNCATE vs DELETE

Command	Category	Table Structure Remains?	Data Deleted?	Speed
DELETE	DML	Yes	Yes (Selected or all rows)	Slow (Logs row-by-row)
TRUNCATE	DDL	Yes (Table becomes empty)	Yes (All rows)	Very Fast
DROP	DDL	No (Table is destroyed)	Yes (All rows)	Instant